

Scribd, the document hosting site, has ditched its Flash-based PDF viewer for a canvas-based HTML5 implementation. Yes, it's good

More search engine goodness: HTML5 can use standardised data formats to offer even more semantic information to any bot that comes a-knocking

With HTML5, these <div> tags get an overhaul, with names that convey their meaning: <article> for anything that may be construed as useful content, <nav> for the site's navigation bar, <header> and <footer> for, well, the header and footer, and so on. This way, search engines know to focus more on the articles, and not as much on the extraneous faff that is the navigation bar or footer. These are just a few examples; we covered creating HTML5 pages in September 2009 (~Cooking with HTML5-), where we talked about these tags, and all their little friends.

<Moving on, then.> The world's HTML5

You're probably wondering why nobody's been talking about the semantic benefits of HTML5. Are we not all questing for a Better Web™? The "problem" with HTML5's semantic tags, unfortunately, is that you can't see them. It may help the web function better, but still look the same.

And that's a problem in marketing departments. Invisible changes that are trying to weave a whole new fabric of the Web? Meh. Rounded corners and shiny buttons? Yippee!

And so the term HTML5 is now a catch-all for a "family of technologies", namely HTML5 the markup language, CSS3, and JavaScript.

And what a family it is. HTML5 is the dad, on whom everyone depends — giving CSS3 and JavaScript the tools they need to let their specialness shine through. CSS3 is the neurotic mum, cleaning and polishing everything so it all looks pretty when the guests come in, and JavaScript is the redeemed little brat — unmanageable and quite useless at first, but now all grown up, and hangs out with the other cool languages.

All right, so it's a stupid analogy. That said, this family of technologies does earn some of those tongue-baths that writers have been lavishing upon it. Let's address the big green elephant in the room first.

<The Flash-killer(?)>

The second part of HTML5 that everyone hopes will put Flash in the ground forever is the <canvas> element (the first

being <video>, of course). In the HTML5 code, it isn't much — just an area that tells the browser, "This is a canvas. Someone will draw on it."

And that Someone is JavaScript. The HTML5 specification provides a bunch of application programming interfaces (APIs) that allow developers to use JavaScript to draw on that canvas. "Draw", there, is a loose term: developers could draw something on the canvas at 50 frames per second, for instance, making movies. The Flash animations of yesterday could thus become the JavaScript-powered canvas drawings of tomorrow, if you will. To see what canvas madness people have come up with, head over to www.canvasdemos.com.

Whether this combination will kill Flash still remains to be seen. Still, we



If Apple's been listening to the feedback, you may not see this message any more. But here it is, preserved for posterity — the message you get when you try to view Apple's HTML5 demos in Chrome

can't but help imagine a future where we aren't listening to our laptop fans go nuts as soon as the Flash plugin kicks into action.

<And that's not all. > The extended family

Have you noticed that you can somewhat use Gmail and Google Docs even when your internet connection is playing coy? That's because both sites use HTML5's Offline Application Caching API, which lets web applications store data on your PC, making them run faster, or remain accessible even when you're offline.

Then there's Geolocation, which lets web applications access your cur-

rent location (with your permission, of course), and deliver location-sensitive content, whatever that might be.

But really, let's get back to that Flash-killing issue again.

<Flash-killer, II>

The short answer is no. HTML5 won't kill Flash — at least not as fast as Apple would have you believe. Firstly, with Flash, nobody has to reach a consensus on a video format — if you can write a Flash player, you can write it to pick up any video format and play it as Flash video. With HTML5, on the other hand, browser developers are still arguing over which format gets supported inside a <video> tag by default — will it be the Apple-backed H.264, Google's WebM, or the open source evangelist's favourite, Ogg? While everyone else waffles on the issue, Flash will remain where it is.

Secondly, you can't view videos in full-screen. You will eventually (one hopes), but right now, you must settle for whatever size the video's creator displays it as.

Finally — and this is crucial — HTML5 video doesn't support ads. YouTube gets its money from ads that appear before or overlaid on videos, and sites like Hulu would fail instantly were it not for the mini commercial breaks within programmes.

For video then, the future isn't now. It's still the future. The canvas element though, has a much better chance against Flash animations — it may be harder to create those animations in JavaScript, but in all probability, we'll see tools that make it easier.

This battle looks even, for now.

<Endnote>

So, what have we learned today? We now know that any talk of HTML5 outside of the web design community must be taken with massive piles of salt, for one. For two, HTML5 may bring the semantic web closer to reality, so we won't look like idiots in four years. For three, video is not the only feather in the HTML5 cap. And finally, while the HTML5 family may substitute Flash in a few places, it isn't quite the killer everyone pretends it is. 